

A
Project Report
On

**StayFinder – Smart Hostel & Room
Finder System**

FOR

Sumago Infotech Pvt. Ltd

IN PARTIAL FULFILMENT
OF
MASTER OF COMPUTER APPLICATIONS



SAVITRIBAI PHULE PUNE UNIVERSITY

SUBMITTED BY
Mr. Raj Singh

UNDER THE GUIDANCE OF
Dr. Dhanashree Patil

INDIRA COLLEGE OF ENGINEERING AND MANAGEMENT, PUNE
(An Autonomous Institute Affiliated to Savitribai Phule Pune University Pune)
Parandwadi, Pune – 410506

2025-2026



(An ISO 9001:2015 Company)

Sumago Infotech Pvt. Ltd.

Strive With Technology...!

Ref No : SIPL/INTERN/2025/1712291

Date:19/12/2025

To,

Raj Abhay Singh

Plot No.- 07, Sandhya Niwas, Ramkrushna Nagar, Chunchale, Nashik.

INTERNSHIP LETTER

Dear Raj,

Sumago Infotech is pleased to offer you an Educational Internship Opportunity as a **Trainee MERN Stack Developer** based on your interview and qualification.

This internship will provide you with hands-on experience, exposure to real-world projects, and an opportunity to enhance your skills in a professional environment.

Additionally, you shall receive multiple learnings as a part of your internship like development of Technical, Managerial, Time Management and Team Management Skills. Additionally you can also grab Performance Rewards and full time employment opportunity during or after the internship based on your performance.

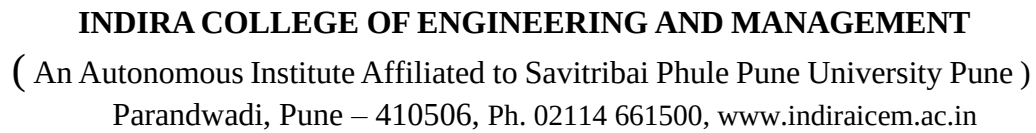
Your internship will conclude after completing 6 Months, commencing from **1st January 2026** and concluding on **1st June 2026**.

We look forward to having you as a part of our team and hope this internship will be a valuable step in your professional journey.

Congratulations and welcome to the team....!



Mrs. Sonali Adhav – Gorade
CEO



Date: -

..... Submitted to the Department of MCA, Indira College of Engineering and Management in partial fulfillment of the requirement for the award of The degree of MASTER OF COMPUTER APPLICATIONS (MCA Affiliated to Savitribai Phule Pune University), is an original work carried out by

No Under my guidance. The matter embodied in this project is a genuine work done by the student and has not been submitted whether to this Organization or to any other University/Organization for the fulfillment of the requirement of any course of study.

Dr. Dhanashree Patil
Project Guide-SYMCA

Dr. Nilesh Uke
Principal-ICEM



INDIRA COLLEGE OF ENGINEERING AND MANAGEMENT
(An Autonomous Institute Affiliated to Savitribai Phule Pune University Pune)
Parandwadi, Pune – 410506, Ph. 02114 661500, www.indiraicem.ac.in

CERTIFICATE

This is to certify that the project report entitled.....

.....

...

Submitted to the Department of Computer Application, Indira College of Engineering and

Management in partial fulfillment of the requirement for the award of the

Degree of MASTER OF COMPUTER APPLICATIONS (MCA affiliated to Savitribai Phule Pune University)

Is an original work carried out by Mr /Ms.....

.....Exam Seat No.....

The matter embodied in this project is a genuine work done by the student and has been Certified
by the following examiners deputed by Savitribai Phule Pune University.

Internal Examiner

External Examiner

Acknowledgment

I would like to express my sincere gratitude to all those who have supported and guided me throughout the successful completion of my project titled “**StayFinder – Smart Hostel & Room Finder System.**”

I am deeply grateful to **Dr. Awantika Bijwe, Head of the MCA Department**, for her constant encouragement, valuable guidance, and continuous support throughout the project.

I would like to extend my heartfelt thanks to my **project guide, Dr. Dhanashree Patil**, for her valuable suggestions, constructive feedback, and consistent motivation at every stage of the project. Her guidance played a crucial role in shaping the direction and quality of this work.

I would also like to express my sincere appreciation to **Sumago Infotech Pvt. Ltd.** for providing me with the opportunity to work as a **Trainee MERN Stack Developer**. The practical exposure and real-world experience gained during the internship were instrumental in the successful development of this project.

I would like to thank all the **faculty members and staff of the MCA Department** for their cooperation and support. I am also thankful to my friends and classmates for their encouragement and timely help.

Finally, I express my heartfelt gratitude to my **family** for their constant support, patience, and encouragement throughout this journey. Their motivation has always been a source of strength for me.

Mr. Raj Abhay Singh

Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
CSS	Cascading Style Sheets
RDBMS	Relational Database Management System
ERD	Entity Relationship Diagram
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
AI	Artificial Intelligence
JSON	JavaScript Object Notation
JWT	JSON Web Token
MCA	Master of Computer Applications
MVC	Model View Controller
URL	Uniform Resource Locator
PWA	Progressive Web Application
RBAC	Role-Based Access Control
RDBMS	Relational Database Management System
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator

Table of Content

Chapter		Content	Page No.
1		Introduction	1
	1.1	Company Profile / Client Profile	1
	1.2	Abstract	2
	1.3	Existing System and Need for System	3
	1.4	Scope of the System	4
	1.5	Operating Environment – Hardware and Software	6
	1.6	Brief Description of Technology Used	7
2		Proposed System	8
	2.1	Study of Similar Systems	8
	2.2	Feasibility Study	9
	2.3	Objectives of the Proposed System	10
	2.4	Users of the System	12
3		Analysis and Design	13
	3.1	System Requirements (Functional and Non-Functional)	14
	3.2	Entity Relationship Diagram (ERD)	16
	3.3	Table Structure	19
	3.4	Use Case Diagram	21
	3.5	Class Diagram	23
	3.6	Activity Diagram	25
	3.7	Deployment Diagram	27

Table of Content

	3.8	Module Hierarchy Diagram	28
	3.9	Input and Output Screens	31
4		Coding	32
	4.1	Code Snippets	33
5		Testing	34
	5.1	Test Strategy	34
	5.2	Unit Test Plan	35
	5.3	Acceptance Test Plan	36
	5.4	Test Cases / Test Scripts	37
6		Limitations of Proposed System	38
7		Proposed Enhancements	39
8		Conclusion	40
9		Bibliography	41

INTRODUCTION

1.1 Company Profile / Institute Profile

Sumago Infotech Pvt. Ltd. is a well-established and rapidly growing IT organization that focuses on delivering innovative and high-quality software solutions. The company provides services in web application development, mobile application development, IT consulting, and technical training programs. It aims to bridge the gap between academic knowledge and industry requirements by offering internship opportunities and real-time project exposure to students. The organization plays a significant role in enhancing technical, managerial, and problem-solving skills among trainees.

The company operates from multiple locations, including Nashik (Head Office), Pune, and California (USA). It is committed to delivering scalable, secure, and efficient software solutions to meet modern business needs.

Vision:

To become a leading global IT organization by delivering innovative, reliable, and technology-driven solutions that empower businesses and individuals.

Mission:

To provide high-quality software solutions, train students with real-world exposure, develop scalable and user-friendly applications, and ensure customer satisfaction through continuous improvement.

During my internship at **Sumago Infotech Pvt. Ltd.**, I worked as a **Trainee MERN Stack Developer**, where I gained hands-on experience in full-stack development and real-world project implementation. As part of this internship, I developed a project titled “**StayFinder – Smart Hostel & Room Finder System**”, which helped me enhance my technical knowledge, problem-solving ability, and teamwork skills.

1.2 Abstract

The rapid advancement of digital technology has significantly transformed the way people search for accommodation and rental properties. Traditional methods of finding hostels or rental rooms are often time-consuming, inefficient, and lack transparency in pricing and availability. To overcome these challenges, the project titled “**StayFinder – Smart Hostel & Room Finder System**” has been developed.

This system is a full-stack web application built using modern technologies such as the **MERN stack (MongoDB, Express.js, React.js, Node.js)** along with Artificial Intelligence integration. The primary objective of the system is to provide a centralized platform where users can search, compare, and book hostels or rental rooms efficiently.

The application includes key features such as user authentication, advanced property search and filtering, real-time communication between tenants and property owners, and secure online payment integration. One of the major highlights of the system is the implementation of an AI-based rent prediction model using the Random Forest algorithm, which helps users determine whether a property is fairly priced, overpriced, or a good deal.

Additionally, the system provides a roommate matching feature that suggests compatible roommates based on user preferences and lifestyle habits. The platform also supports automated generation of rental agreements in PDF format after successful payment, ensuring a smooth and professional transaction process. Real-time notifications keep users updated about bookings, messages, and price changes.

Overall, the system is designed to be user-friendly, scalable, and efficient, reducing manual effort while improving transparency and user experience in the rental accommodation process.

1.3 Existing System and Need for Proposed System

The existing system for finding rental accommodations has several limitations. There is no centralized platform, as property information is scattered across multiple sources, making it difficult for users to compare options efficiently. The process is time-consuming because users often need to visit multiple locations or contact different agents. Additionally, there is a lack of price transparency, which may lead to overpricing or unfair deals.

Communication between tenants and property owners is also limited, as there is no real-time interaction system, resulting in delays in decision-making. Rental agreements are typically created manually, increasing the chances of errors and inconsistencies. Furthermore, existing systems do not provide intelligent recommendations or personalized suggestions, and there is no support for roommate matching.

To address these issues, the proposed system introduces a centralized and intelligent platform that simplifies the entire process. It enables users to search, filter, and book properties online, saving time and effort. The integration of AI-based rent prediction ensures price transparency and helps users make better decisions. Real-time chat functionality allows direct communication between tenants and property owners.

The system also automates rental agreement generation in PDF format, reducing manual work and improving accuracy. Smart notifications keep users informed about important updates, while the roommate matching feature enhances user experience by suggesting compatible roommates. Secure online payment integration ensures safe and reliable transactions.

1.4 Scope of the System

The **StayFinder – Smart Hostel & Room Finder System** is designed to provide a comprehensive, centralized, and intelligent platform for managing rental accommodations. The scope of the system covers both user-side functionalities and administrative operations, ensuring a smooth and efficient experience for all stakeholders.

The system enables **online property listing and browsing**, where property owners can add detailed information about their rooms or hostels, including price, location, amenities, and images. Users can easily explore available properties from any location using an internet-enabled device.

It provides **advanced search and filtering features**, allowing users to find suitable accommodations based on parameters such as city, price range, room type, and available facilities. This significantly reduces the time and effort required to search for properties.

The platform includes a secure **user registration and authentication system**, ensuring safe access to user accounts and personalized services. It also integrates an **AI-based rent prediction system**, which helps users analyze whether a property is fairly priced, overpriced, or a good deal.

A **real-time communication system** is implemented using chat functionality, enabling direct interaction between tenants and property owners. This improves decision-making and reduces communication delays.

The system also supports **online booking and secure payment integration**, allowing users to book properties and complete transactions seamlessly. After successful payment, an **automated rental agreement** is generated in PDF format, reducing manual paperwork and ensuring standardization.

Additionally, the platform offers a **roommate matching system**, which suggests compatible roommates based on user preferences, lifestyle, and habits. A **notification and alert system** keeps users informed about booking updates, messages, and price changes.

An **admin dashboard** is also included, enabling efficient management of users, properties, bookings, and overall system activities.

1.5 Operating Environment – Hardware and Software

Hardware Requirements

The system does not require high-end hardware and can function effectively on standard computing devices. The recommended hardware configuration is as follows:

- **Processor:** Minimum **2.0 GHz dual-core processor** or higher
- **RAM:** Minimum **4 GB RAM** (8 GB recommended for better performance)
- **Storage:** At least **20 GB free disk space**
- **Client Devices:** Desktop, Laptop, Tablet, or Smartphone
- **Network:** **Stable internet connection** for real-time features and communication

Software Requirements

The system is built using modern web technologies and requires the following software components:

Frontend Technologies

- **React.js** – for building user interface
- **Vite** – for fast development and build process
- **HTML, CSS, JavaScript** – for designing and styling

Backend Technologies

- **Node.js** – runtime environment
- **Express.js** – backend framework for API development

Database

- **MongoDB** – NoSQL database for storing application data
- **Mongoose** – for database modeling and interaction

AI/ML Technologies

- **Python (FastAPI)** – for AI model deployment
- **Scikit-learn** – for implementing **Random Forest algorithm** (rent prediction model)

Other Tools and Integrations

- **Socket.io** – for real-time communication (chat and notifications)
- **JWT (JSON Web Token)** – for secure authentication
- **Cloudinary** – for image storage and management
- **Razorpay** – for secure online payment integration
- **PDFKit** – for generating rental agreements in PDF format
- **Nodemailer** – for sending email notifications
- **Google Maps API / OpenStreetMap** – for location-based services

Operating System Support

- **Windows, Linux, macOS** – for development and deployment
- **Android and iOS** – for accessing the application via mobile devices

1.6 Brief Description of Technology Used

1.6.1 Frontend Technologies

The frontend of the system is developed using **React.js**, which enables the creation of dynamic and reusable user interface components. It enhances performance through efficient rendering and supports a component-based architecture.

1.6.2 Backend Technologies

The backend is developed using **Node.js**, which allows efficient handling of multiple client requests. **Express.js** is used as a lightweight framework to build RESTful APIs and manage server-side logic.

1.6.3 Database Technologies

The system uses **MongoDB**, a NoSQL database that stores data in a flexible, JSON-like format. It is highly scalable and suitable for handling large volumes of application data.

1.6.4 Artificial Intelligence Technologies

Artificial Intelligence plays a key role in the system. **Python (FastAPI)** is used to develop high-performance APIs for AI model integration. The system uses **Scikit-learn** to implement the **Random Forest algorithm**, which predicts rental prices based on various input factors such as location, amenities, and room features.

1.6.5 Real-Time Communication

The system uses **Socket.io** to enable real-time communication between users. This technology supports instant messaging and live notifications, enhancing user interaction.

1.6.6 Third-Party Integrations

The system integrates several third-party tools to extend its functionality. **Cloudinary** is used for storing and managing images efficiently. **Razorpay** is integrated for secure online payment processing. **PDFKit** is used to generate rental agreements in PDF format, and **Nodemailer** is used to send email notifications such as booking confirmations and alerts.

PROPOSED SYSTEM

2.1 Study of Similar Systems

The development of the **StayFinder – Smart Hostel & Room Finder System** is based on an analysis of existing platforms that provide property rental and accommodation services. Studying these systems helps in identifying their strengths and limitations, which guides the design of a more efficient and advanced solution.

One of the widely used platforms is **MagicBricks**, which provides a large database of property listings along with search filters based on location, budget, and property type. It allows users to view property details and connect with agents or brokers. However, the platform has certain limitations such as dependency on brokers, lack of real-time communication, absence of AI-based price prediction, and no support for roommate matching.

Another popular platform is **Airbnb**, which connects property owners with users looking for short-term stays. It offers an easy booking system, a wide variety of property options, and user reviews. Despite its popularity, it mainly focuses on short-term rentals and lacks features for long-term accommodation management, advanced AI-based recommendations, and roommate matching.

Other platforms such as **NoBroker**, **Housing.com**, and **99acres** also provide property search services. These platforms offer multiple listings and filtering options but still face challenges such as outdated or duplicate listings, limited automation, lack of advanced AI features, and absence of real-time communication systems.

From the study of these systems, it is observed that while they provide basic functionalities, they lack intelligent features such as **AI-based rent prediction**, **real-time communication**, **automated agreement generation**, and **roommate matching systems**. These limitations highlight the need for a more advanced and integrated solution.

2.2 Feasibility Study

The feasibility study evaluates whether the proposed system can be developed and implemented successfully. It includes **technical feasibility**, **economic feasibility**, and **operational feasibility**.

Technical feasibility focuses on the availability of technology and resources required to develop the system. The StayFinder system is built using modern and widely used technologies such as **React.js, Node.js, Express.js, MongoDB, and Python (FastAPI)**. These technologies are open-source, well-documented, and suitable for building scalable applications. The integration of AI-based rent prediction using the **Random Forest algorithm** is feasible with tools like Scikit-learn. Real-time features such as chat and notifications are implemented using **Socket.io**, ensuring efficient communication.

Economic feasibility evaluates the cost involved in developing and maintaining the system. The use of open-source technologies reduces licensing costs, and no expensive hardware is required. Third-party services such as **Cloudinary** and **Razorpay** offer cost-effective solutions. Since the project is developed during an internship, the overall development cost is minimal, making it economically viable.

Operational feasibility determines how effectively the system can be used in real-world scenarios. The system provides a **user-friendly interface**, allowing users to perform tasks such as searching properties, booking rooms, and making payments easily. It automates key processes like agreement generation and notifications, reducing manual effort. The system is suitable for students, working professionals, and property owners, making it practical and widely usable.

2.3 Objectives of the Proposed System

The primary objective of the **StayFinder – Smart Hostel & Room Finder System** is to develop a smart, efficient, and user-friendly platform that simplifies the process of finding and managing rental accommodations.

The system aims to provide a **centralized platform** where users can search, compare, and book properties easily. It enables **efficient property search** through advanced filters such as location, price range, room type, and amenities.

A key objective is to implement **AI-based rent prediction**, which helps users determine whether a property is fairly priced, overpriced, or a good deal. This improves transparency and supports better decision-making.

The system also focuses on enabling **real-time communication** between tenants and property owners through a chat system, reducing delays and improving interaction.

Another important objective is to provide **secure online booking and payment integration**, ensuring smooth and reliable transactions. The system also automates the process of generating **rental agreements in PDF format**, reducing manual errors and paperwork.

Additionally, the system includes a **roommate matching feature**, which helps users find compatible roommates based on preferences such as budget, lifestyle, and habits.

Secondary objectives include improving **user experience**, ensuring **data security**, providing **real-time notifications**, supporting **scalability**, and reducing manual effort through automation.

2.4 Users of the System

1. Tenants (Customers/Users)

Tenants are the primary users of the system who are looking for hostels or rental rooms.

Functions:

- **Register and login** to the system
- Search properties using **advanced filters**
- View detailed property information (price, location, amenities)
- Use **AI-based rent prediction** to evaluate pricing
- Communicate with property owners using **real-time chat**
- Book rooms and make **secure online payments**
- Receive **notifications and alerts**
- View booking history and download **rental agreements**
- Use the **roommate matching system** to find compatible roommates

2. Property Owners

Property owners are users who list their properties on the platform for rent.

Functions:

- Register and manage their profile
- Add, update, and delete property listings
- Upload property details such as price, images, and amenities
- Respond to tenant queries via **real-time chat**
- Approve or reject booking requests
- Track bookings and payments

- Receive notifications about user activities

3. Admin (System Administrator)

The admin is responsible for managing and controlling the overall system.

Functions:

- Manage users (tenants and owners)
- Monitor and control property listings
- View and manage bookings and transactions
- Handle complaints and system issues
- Generate reports and analyze system performance
- Ensure **security and proper functioning** of the platform

4. Roommate Seekers

These are users who specifically use the system to find compatible roommates.

Functions:

- Create and manage roommate profiles
- Set preferences such as budget, lifestyle, habits
- View and match with **compatible roommates**
- Communicate with matched users

Analysis and Design

3.1 System Requirements

Functional Requirements

Functional requirements describe the **core operations and features** that the system must perform.

1. User Authentication and Management

- Users should be able to **register and login securely**.
- The system should support **role-based access** (Tenant, Owner, Admin).
- Users should be able to **update their profile information**.

2. Property Search and Filtering

- Users should be able to **search properties** based on location, price, and type.
- The system should provide **advanced filtering options** for better results.

4. AI-Based Rent Prediction

- The system should analyze property data and provide **AI-based price prediction**.
- It should classify properties as **overpriced, fair, or best deal**.

3. Booking System

- Users should be able to **book properties online**.
- The system should store booking details and maintain **booking history**.
- Owners should be able to **approve or reject bookings**.

4. Payment Integration

- The system should support **secure online payments**.
- Payment status should be recorded and updated properly.

5. Rental Agreement Generation

- The system should automatically generate a **PDF rental agreement** after successful payment.
- Users should be able to **download the agreement**.

6. Roommate Matching System

- Users should be able to create a **roommate profile**.
- The system should suggest **compatible roommates** based on preferences.

7. Admin Control

- Admin should manage users, properties, bookings, and reports.
- Admin should monitor system activities and ensure smooth functioning.

Non-Functional Requirements

1. Performance

- The system should provide **fast response time**.
- It should handle multiple users simultaneously without delays.

2. Security

- User data should be protected using **authentication and encryption techniques**.
- Secure payment processing must be ensured.

3. Scalability

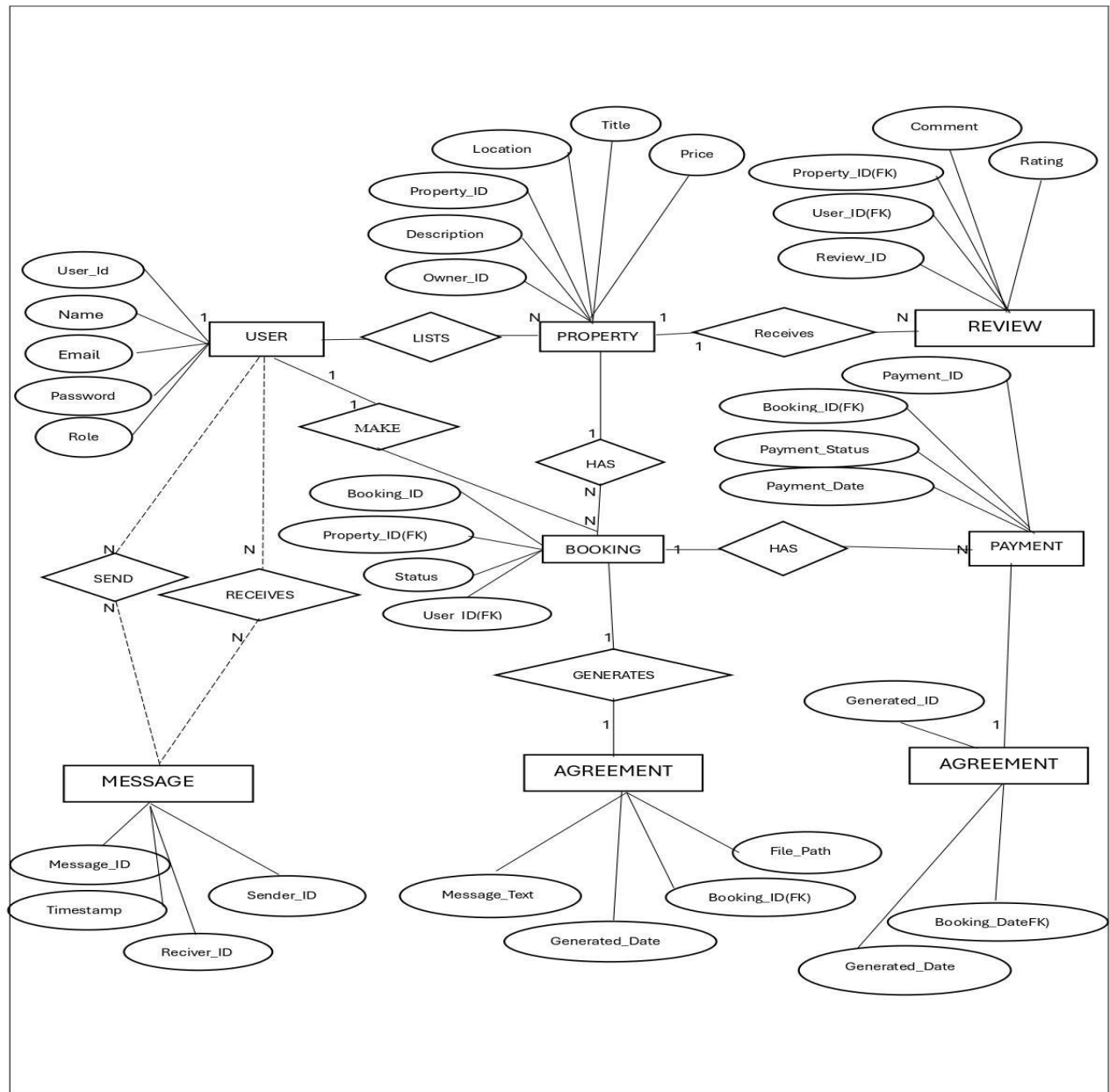
- The system should be able to **handle increasing users and data** without performance issues.

4. Usability

- The system should have a **simple and user-friendly interface**.
- Users should be able to navigate easily without technical knowledge.

3.2 Entity Relationship Diagram (ERD)

Diagram



The **Entity Relationship Diagram (ERD)** represents the structure of the database and shows the relationships between different entities in the system. The main entity in the system is the

User, which stores information about all users, including tenants, property owners, and administrators. It contains attributes such as user ID, name, email, password, and role.

The **Property** entity contains details about properties listed by owners, including title, location, price, description, amenities, and owner ID.

The **Booking** entity stores information about bookings made by users. It includes attributes such as booking ID, user ID, property ID, booking date, and status.

The **Payment** entity maintains records of transactions, including payment ID, booking ID, amount, payment status, and payment date.

The **Review** entity stores user feedback for properties, including rating and comments.

The **Message** entity supports the chat system by storing messages exchanged between users, along with sender, receiver, and timestamp details.

The **Agreement** entity stores rental agreement details generated after successful payment, including file path and generation date.

These entities are interconnected through relationships such as user-to-property, property-to-booking, booking-to-payment, and user-to-message, forming a structured and efficient database design.

3.3 Table Structure

The table structure defines how data is organized and stored in the database of the **StayFinder – Smart Hostel & Room Finder System**. The system uses a **MongoDB (NoSQL) database**, but for better understanding, the structure is represented in a tabular (relational) format.

1. User Table

Field Name	Data Type	Description
user_id	INT (PK)	Unique identifier for each user
name	VARCHAR	Name of the user
email	VARCHAR	Unique email address of the user
password	VARCHAR	Encrypted password for security
role	VARCHAR	Role of the user (Tenant / Owner / Admin)

2. Property Table

Field Name	Data Type	Description
property_id	INT (PK)	Unique property ID
title	VARCHAR	Property title
location	VARCHAR	Property location
price	INT	Rent price
description	TEXT	Property details
amenities	TEXT	Facilities available
owner_id	INT (FK)	Linked to User table

3. Booking Table

Field Name	Data Type	Description
booking_id	INT (PK)	Unique booking ID
user_id	INT (FK)	User who booked
property_id	INT (FK)	Booked property
booking_date	DATE	Date of booking
status	VARCHAR	Booking status

4. Payment Table

Field Name	Data Type	Description
payment_id	INT (PK)	Unique payment ID
booking_id	INT (FK)	Related booking
amount	INT	Payment amount
payment_status	VARCHAR	Status (Paid/Pending)
payment_date	DATE	Date of payment

5. Review Table

Field Name	Data Type	Description
review_id	INT (PK)	Unique review ID
user_id	INT (FK)	User who reviewed
property_id	INT (FK)	Reviewed property
rating	INT	Rating (1–5)
comment	TEXT	Review message

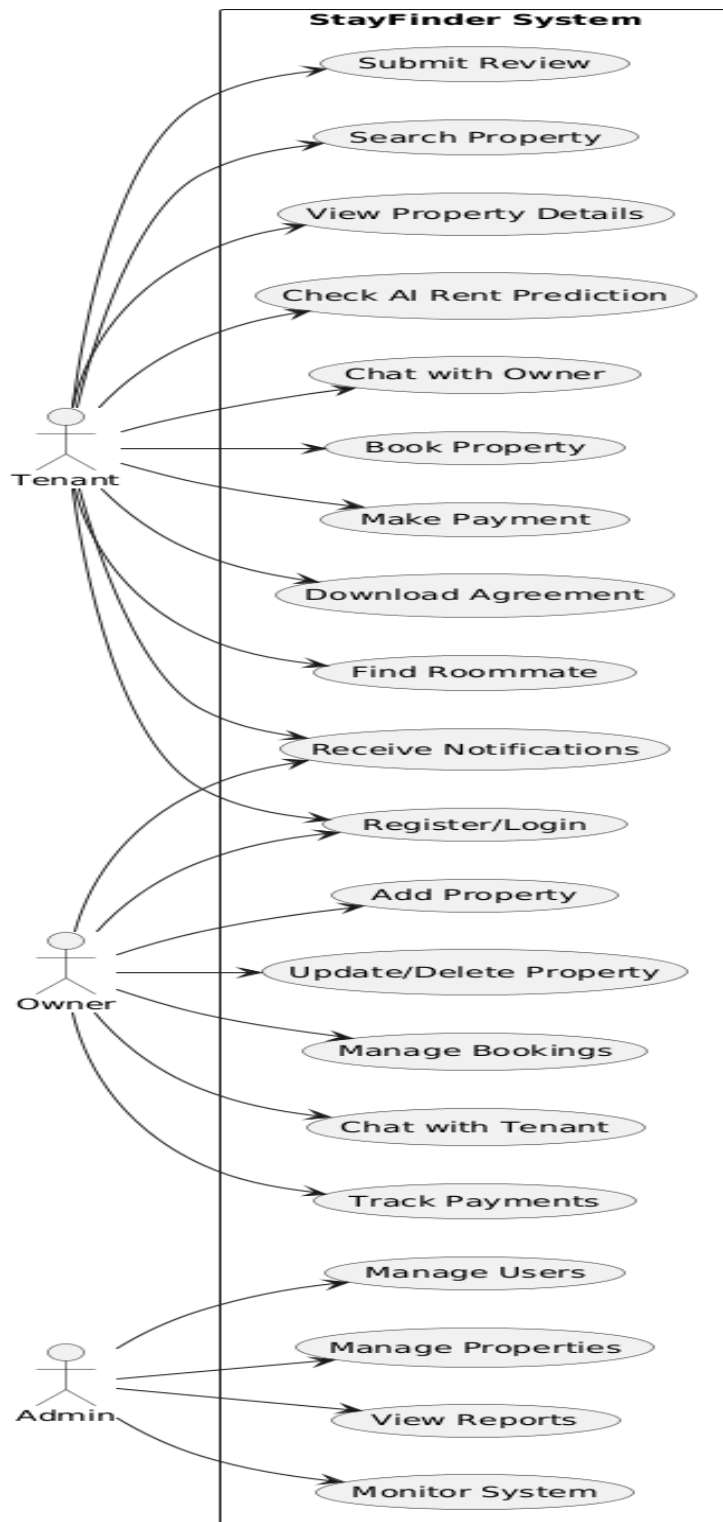
6. Message Table

Field Name	Data Type	Description
message_id	INT (PK)	Unique message ID
sender_id	INT (FK)	Sender user
receiver_id	INT (FK)	Receiver user
message_text	TEXT	Message content
timestamp	DATETIME	Message time

7. Agreement Table

Field Name	Data Type	Description
agreement_id	INT (PK)	Unique agreement ID
booking_id	INT (FK)	Related booking
file_path	VARCHAR	PDF file location
generated_date	DATE	Agreement date

3.4 Use Case Diagram



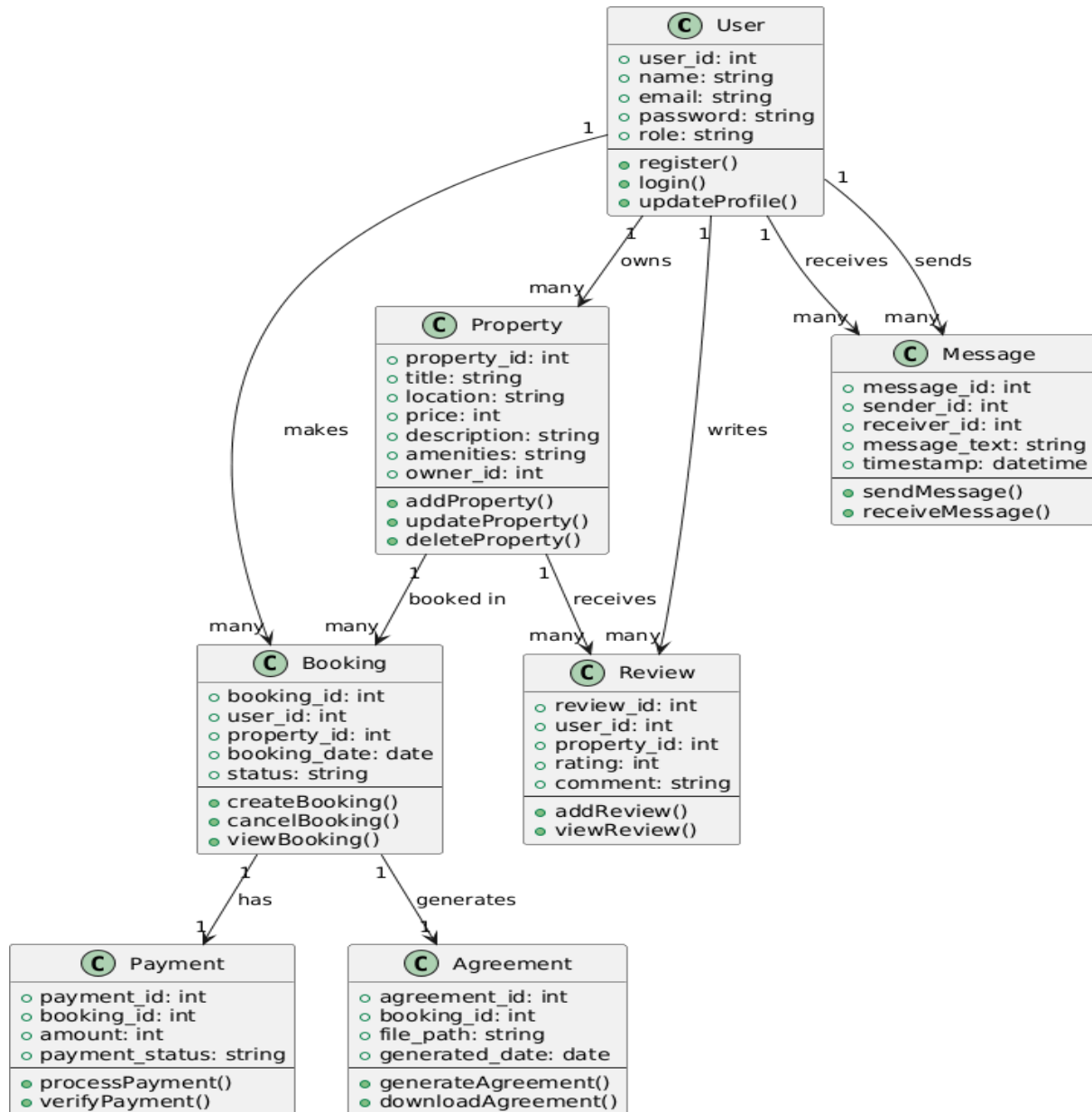
The **Use Case Diagram** illustrates the interaction between users and the system.

The system includes three main actors: **Tenant**, **Property Owner**, and **Admin**. The tenant can perform actions such as registration, login, property search, viewing details, checking AI-based rent prediction, chatting with owners, booking properties, making payments, downloading agreements, receiving notifications, finding roommates, and submitting reviews.

The property owner can manage property listings by adding, updating, or deleting properties, respond to user queries, handle booking requests, track payments, and receive notifications.

The admin is responsible for managing users, monitoring properties, controlling bookings and transactions, generating reports, and maintaining system security.

3.5 Class Diagram



The **Class Diagram** represents the structure of the system in terms of classes and their relationships.

The **User class** includes attributes such as user ID, name, email, password, and role, along with methods like register, login, and update profile.

The **Property class** manages property-related data and includes methods for adding, updating, and deleting properties.

The **Booking class** handles booking operations such as creating, canceling, and viewing bookings.

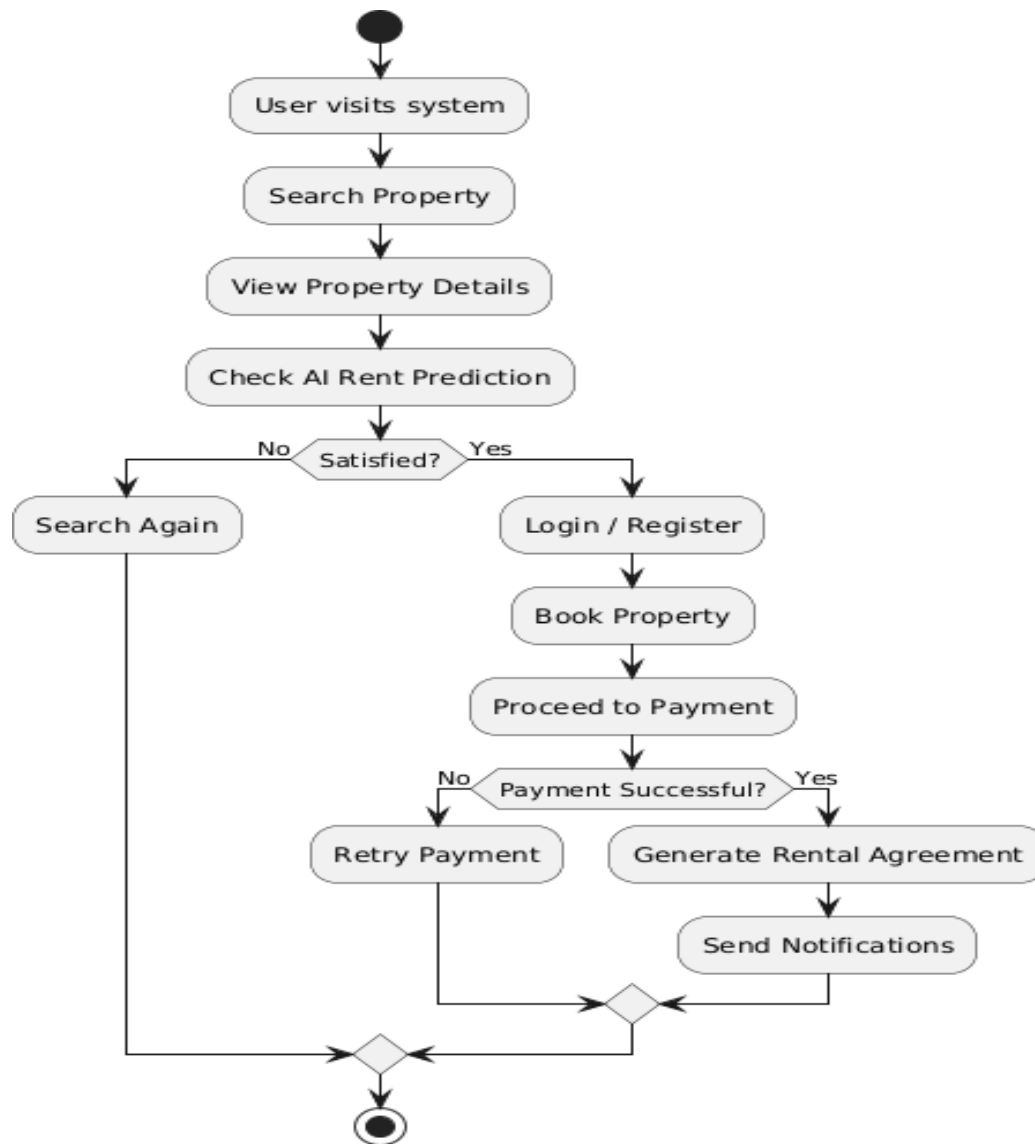
The **Payment class** manages transactions, including processing and verifying payments.

The **Review class** stores user feedback and provides methods for adding and viewing reviews.

The **Message class** supports communication between users through sending and receiving messages.

The **Agreement class** handles the generation and downloading of rental agreements.

3.6 Activity Diagram



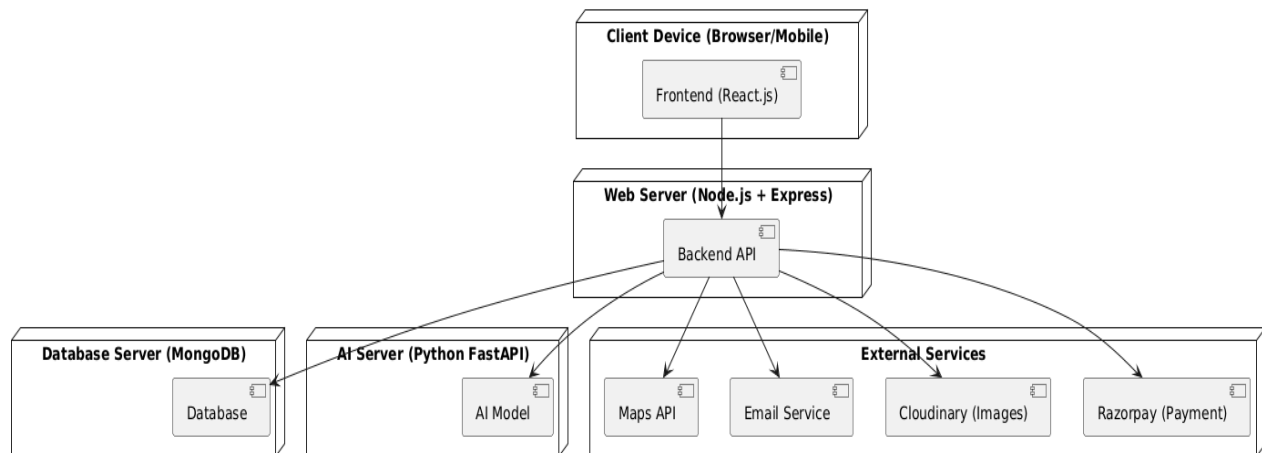
Process Flow of the System

The main activities involved in the system are as follows:

1. **Start**
2. User visits the system
3. **Search for property** using filters

4. View property details
5. Check AI-based rent prediction
6. Decide whether to proceed
7. If not satisfied → return to search
8. If satisfied → proceed to login/register
9. **Login or Register**
10. **Book the property**
11. Proceed to **payment process**
12. Check payment status
13. If payment fails → retry payment
14. If payment is successful →
15. Generate rental agreement (PDF)
16. Send notifications to user
17. End process

3.7 Deployment Diagram



Components of the System

The system consists of the following main components:

1. Client Device

- Represents users accessing the system through **web browsers or mobile devices**.
- Users can perform activities such as searching properties, booking rooms, chatting, and making payments.
- Technologies used: **React.js (Frontend)**

2. Web Server (Application Server)

- Hosts the backend application developed using **Node.js and Express.js**.
- Handles API requests, authentication, business logic, and communication between frontend and database.
- Manages features such as booking, chat, notifications, and payments.

3. Database Server

- Stores all application data such as users, properties, bookings, payments, and reviews.
- Implemented using **MongoDB**.

4. AI Server

- Responsible for handling **AI-based rent prediction**.
- Developed using **Python (FastAPI)**.
- Processes input data and returns predicted rental prices.

5. External Services

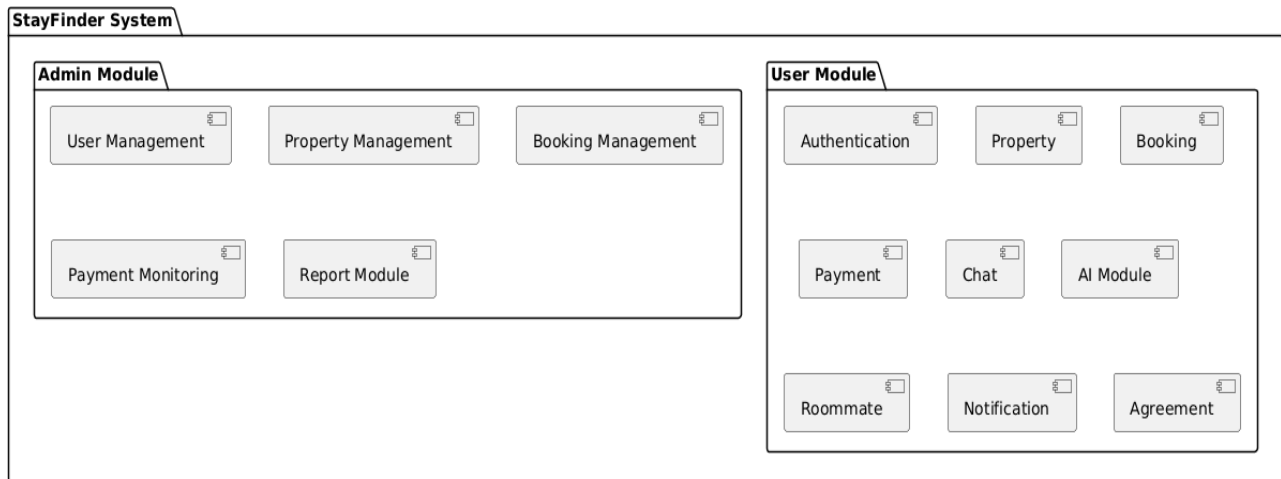
The system integrates with external services for additional functionalities:

- **Payment Gateway (Razorpay)** – for secure online transactions
- **Cloudinary** – for storing property images
- **Email Service (Nodemailer/Gmail)** – for sending notifications
- **Maps API (Google Maps/OpenStreetMap)** – for location services

Working of Deployment

- The **Client Device** sends requests to the **Web Server**.
- The Web Server processes the request and interacts with the **Database Server** to fetch or store data.
- For AI-based features, the Web Server communicates with the **AI Server**.
- External services are used for payments, image storage, and notifications.
- The response is sent back to the client device.

3.8 Module Hierarchy Diagram



The **Module Hierarchy Diagram** represents the structural organization of the system into different modules and sub-modules.

The system is divided into two main modules: the **User Module** and the **Admin Module**.

The **User Module** handles functionalities related to tenants and property owners. It includes several sub-modules such as **authentication**, which manages registration and login; **property module**, which allows users to search and view properties; and **booking module**, which enables users to book properties and view booking history.

The module also includes a **payment module** for handling transactions and payment verification, a **chat module** for real-time messaging, and an **AI module** for rent prediction and price analysis.

Additionally, the **roommate module** helps users find compatible roommates, while the **notification module** manages alerts and updates. The **agreement module** is responsible for generating and downloading rental agreements in PDF format.

The **Admin Module** is responsible for managing the overall system. It includes sub-modules such as **user management**, **property management**, **booking management**, **payment monitoring**, and **report generation**. These modules allow the admin to monitor system activities and ensure smooth operation.

3.9 Input and Output Screens

Registration Page

The screenshot shows the StayFinder web application's registration page. The page has a dark blue header with the StayFinder logo and navigation links: Home, Browse, Roommates, and a Sign In button. The main content area is white and features a 'Create Account' form. The form includes a 'Sign In' link, a 'Register' tab, and a heading 'Create Account' with the subtext 'Join 12,000+ tenants finding smarter rooms'. The form fields are: Full Name (with placeholder 'Your full name'), Email (with placeholder 'you@email.com'), Phone (with placeholder '+91 98765 43210'), and Password (with a minimum length requirement of 6 characters and a toggle for visibility). Below the password field, there are two radio buttons for 'I am a': 'Tenant' (Looking for a room) and 'Owner' (Listing property). A red 'Create Account' button is at the bottom of the form. The page is displayed on a mobile device within a web browser window, with the URL 'localhost:5173/auth' visible in the address bar.

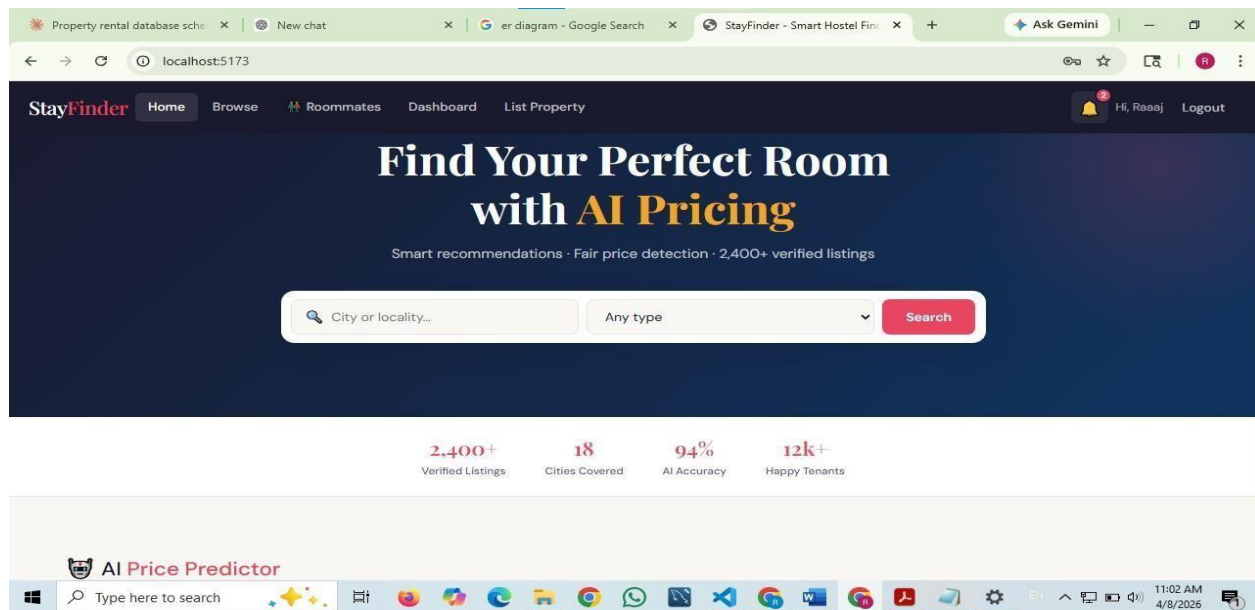
This page is the registration screen of the **StayFinder web application** where new users can create an account. It includes a form to enter basic details like name, email, phone number, and password. Users can also select their role as a Tenant or Owner, which helps the system provide relevant features. A “**Create Account**” button is used to submit the form, and the navigation bar at the top allows access to other sections or the sign-in page.

Login Page:

The screenshot shows the StayFinder web application's login page. The page has a dark blue header with the StayFinder logo and navigation links: Home, Browse, Roommates, and a Sign In button. The main content area is white and features a 'Welcome back' message with the subtext 'Sign in to manage bookings and wishlists'. Below the message, there is a 'Sign in' form with fields for Email Address (with placeholder 'you@email.com') and Password (with a toggle for visibility). A red 'Sign in' button is at the bottom of the form. Below the button, there is a link that says 'Don't have an account? Register free'. The page is displayed on a mobile device within a web browser window, with the URL 'localhost:5173/auth' visible in the address bar.

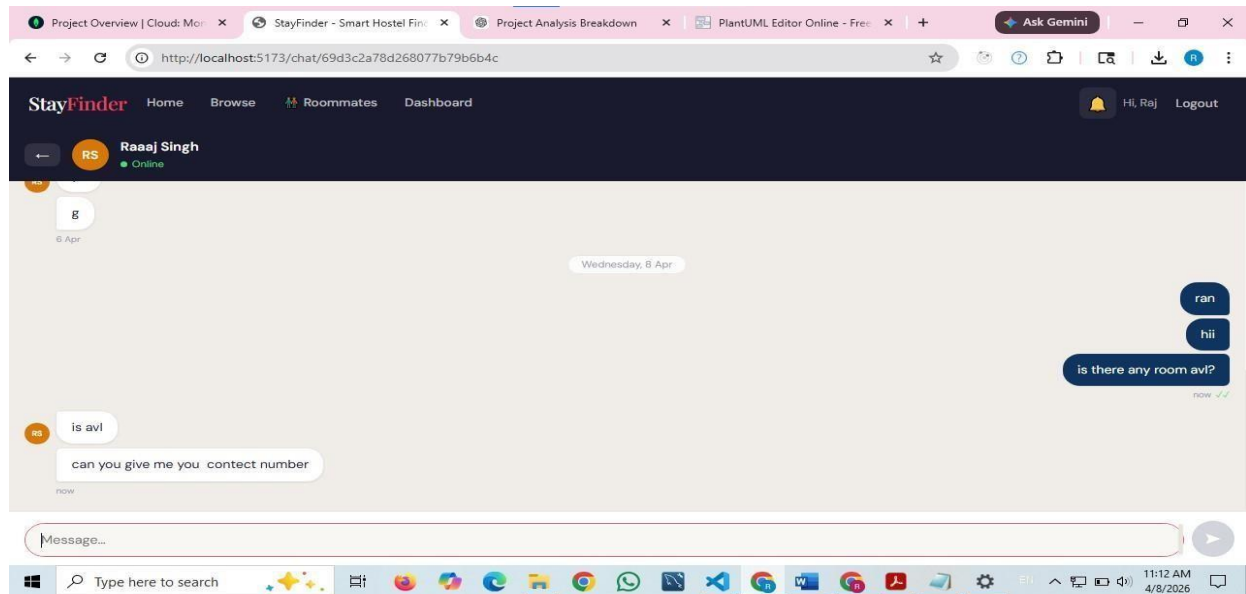
This page is the **Sign In (Login)** screen of the **StayFinder web application**, where existing users can access their accounts. At the top, the **navigation bar** includes links such as **Home, Browse, and Roommates**, along with a **Sign In button** for quick access. In the center, there is an **authentication card** with the **Sign In tab active**, indicating that the user is on the login section. The form allows users to enter their **Email Address** and **Password**, with an option to toggle password visibility using the eye icon. A prominent **Sign In button** is provided to submit the credentials and log into the system. Below the form, there is a small prompt saying “**Don’t have an account? Register free**”, which redirects new users to the registration page.

Home Page:



The Home Page provides an overview of the platform and includes a search bar for finding properties. It highlights key features such as AI-based pricing and displays statistics to build user trust.

Chatting:



This page is the **Chat/Messaging** section of the **StayFinder web application**, where users can communicate directly with each other. It shows a conversation between users with **real-time messages**, including sent and received texts. The interface displays the **user's name**, **online status**, and chat history, while a **message input box** at the bottom allows users to send new messages. This feature helps tenants and owners easily discuss room availability and details.

CODING

4.1 Code Snippets

This section presents the core coding logic used in developing the **StayFinder – Smart Hostel & Room Finder System**. The system is implemented using the **MERN stack (MongoDB, Express.js, React.js, Node.js)** along with **Artificial Intelligence integration** and **real-time communication features**. The implementation is modular and divided into key components such as **authentication, property management, booking system, payment integration, real-time chat**, and **AI-based rent prediction**.

1. User Registration (Backend – Node.js/Express)

```
// Register User
app.post("/api/auth/register", async (req, res) => {
  try {
    const { name, email, password } = req.body;

    const userExists = await User.findOne({ email });
    if (userExists) {
      return res.status(400).json({ message: "User already exists" });
    }

    const user = new User({ name, email, password });
    await user.save();

    res.status(201).json({ message: "Registration successful" });
  } catch (error) {
    res.status(500).json({ message: "Server Error" });
  }
});
```

2. User Login (Authentication with JWT)

```
// Login User
app.post("/api/auth/login", async (req, res) => {
  const { email, password } = req.body;

  const user = await User.findOne({ email });
  if (!user) {
```



```
    return res.status(400).json({ message: "Invalid credentials" });
  }

```

```
const isMatch = await user.matchPassword(password);
if (!isMatch) {
  return res.status(400).json({ message: "Invalid credentials" });
}

```

```
res.json({ message: "Login successful", user });
});

```

3. Payment Integration (Basic Example)

```
// Payment Verification (Mock)
app.post("/api/payment/verify", async (req, res) => {
  const { bookingId } = req.body;

```

```
    const payment = new Payment({
      booking_id: bookingId,
      amount: 5000,
      payment_status: "Paid",
    });

```

```
    await payment.save();

```

```
    res.json({ message: "Payment successful" });
  });

```

4. Real-Time Chat (Socket.io)

```
// Socket.io Chat
io.on("connection", (socket) => {
  socket.on("send_message", (data) => {
    socket.to(data.roomId).emit("receive_message", data);
  });
});

```

TESTING

5.1 Test Strategy

The **test strategy** defines the approach used to ensure that the system functions correctly, efficiently, and securely. The testing process focuses on identifying errors, verifying system functionality, and ensuring that the application meets user requirements.

The system primarily follows **manual testing**, along with validation of different modules and real-time features.

Various testing approaches are used to evaluate the system. **Functional testing** ensures that all features such as registration, login, property search, booking, payment, and chat system work as expected.

User Interface (UI/UX) testing verifies that the system provides a user-friendly and responsive interface across different devices such as mobile, tablet, and desktop.

Integration testing checks the interaction between different modules, including frontend, backend, database, and AI components. It ensures proper data flow and communication between system components.

System testing evaluates the entire system as a whole to ensure that all modules function together without errors.

Performance testing ensures that the system performs efficiently under different conditions and can handle multiple users simultaneously.

Security testing verifies that user data is protected through authentication and encryption techniques, and ensures secure payment processing.

Regression testing ensures that new updates or changes do not affect existing functionalities.

Compatibility testing ensures that the system works correctly across different browsers, devices, and operating systems.

5.2 Unit Test Plan

The **unit test plan** focuses on testing individual modules and components of the system to ensure that each function works correctly.

The system follows a **white-box testing approach**, where the internal logic and code structure are tested. Each module is tested with both valid and invalid inputs.

The **user authentication module** is tested to verify registration, login functionality, password validation, and session handling.

The **property management module** is tested for adding, updating, and deleting property listings, as well as validating property details and image uploads.

The **booking module** is tested to ensure proper booking creation, status updates, and accurate linkage between users and properties.

The **payment module** is tested for successful and failed transactions, ensuring correct payment status updates and secure handling.

The **chat module** is tested to verify real-time message delivery, storage, and retrieval of messages.

The **AI rent prediction module** is tested to ensure accurate input handling and correct prediction outputs.

The **notification module** is tested for generating and delivering alerts related to bookings, messages, and payments.

The **agreement module** is tested to verify automatic generation of PDF agreements and proper download functionality.

Test data includes valid inputs, invalid inputs, and boundary values. A test case is considered successful if the actual output matches the expected output. Any failures are recorded and resolved before moving forward.

5.3 Test Cases / Test Scripts

Test Case Table

Test Case ID	Description	Input	Expected Output	Actual Output	Status
TC01	Load Home Page	Open URL	Homepage loads successfully	As expected	Pass
TC02	User Registration	Valid details	Account created successfully	As expected	Pass
TC03	Registration with existing email	Existing email	Error message displayed	As expected	Pass
TC04	User Login	Valid credentials	Login successful	As expected	Pass
TC05	Login with wrong password	Invalid password	Error message shown	As expected	Pass
TC06	Search Property	Enter location	Relevant properties displayed	As expected	Pass
TC07	View Property Details	Select property	Property details displayed	As expected	Pass
TC08	AI Rent Prediction	Property data	Price prediction shown	As expected	Pass
TC09	Add Property (Owner)	Valid details	Property added successfully	As expected	Pass
TC10	Update Property	Modify details	Updated successfully	As expected	Pass
TC11	Delete Property	Select property	Property removed	As expected	Pass
TC12	Add to Booking	Select property	Booking created	As expected	Pass
TC13	View Booking History	Logged-in user	Booking list displayed	As expected	Pass
TC14	Payment Processing	Valid payment details	Payment successful	As expected	Pass
TC15	Payment Failure	Invalid details	Payment failed message	As expected	Pass
TC16	Generate Agreement	After payment	PDF generated	As expected	Pass

TC17	Download Agreement	Click download	File downloaded	As expected	Pass
TC18	Chat Message Send	Send message	Message delivered	As expected	Pass
TC19	Chat Message Receive	Incoming message	Message displayed	As expected	Pass
TC20	Notification Trigger	Booking update	Notification shown	As expected	Pass
TC21	Roommate Matching	Profile input	Matches displayed	As expected	Pass
TC22	Review Submission	Rating & comment	Review saved	As expected	Pass
TC23	Logout	Click logout	User logged out	As expected	Pass
TC24	Session Timeout	Idle user	Auto logout	As expected	Pass
TC25	Responsive UI	Open on mobile	UI adjusts properly	As expected	Pass
TC26	Search Invalid Property	Random input	“No results found”	As expected	Pass
TC27	Empty Cart Checkout	No booking	Error message shown	As expected	Pass
TC28	Update Profile	Change details	Profile updated	As expected	Pass
TC29	Admin Manage Users	Admin access	Users displayed	As expected	Pass
TC30	Admin View Reports	Open reports	Data displayed	As expected	Pass

LIMITATIONS OF THE PROPOSED SYSTEM

The **StayFinder – Smart Hostel & Room Finder System** provides a modern and efficient solution for rental accommodation; however, it has certain limitations that can be improved in future versions.

The system is highly dependent on a **stable internet connection**, as it is a web-based application. Users with poor connectivity may experience delays or interruptions while using the platform.

The **AI-based rent prediction model** is trained on limited or sample datasets, which may affect the accuracy of predictions in real-world scenarios. The model may not always reflect actual market conditions.

Currently, the system does not provide a **dedicated mobile application** for Android or iOS. Although it is responsive, a native mobile app could enhance user experience and accessibility.

The system supports limited **payment gateway options**, which may restrict flexibility for users who prefer different payment methods, especially international users.

It does not fully support a **multi-vendor marketplace**, where multiple property owners can independently manage their dashboards at scale.

The platform provides only **basic reporting and analytics**, lacking advanced features such as real-time dashboards, predictive insights, and graphical reports.

There are limited options for **customization**, as users and property owners cannot extensively modify the interface or system behavior according to their preferences.

Although standard security mechanisms are implemented, the system may require more advanced features such as **multi-factor authentication** and enhanced encryption for higher-level security.

The system also lacks **offline functionality**, as all operations require real-time connectivity with the server.

PROPOSED ENHANCEMENTS

To improve the functionality and scalability of the system, several enhancements can be implemented in future versions.

A dedicated **mobile application** for Android and iOS can be developed to improve accessibility, performance, and user engagement. Features such as push notifications can further enhance usability.

The system can be enhanced with **advanced AI and Machine Learning algorithms** to provide more accurate rent predictions, personalized property recommendations, and user behavior analysis.

Support for a **multi-vendor marketplace** can be added, allowing multiple property owners to manage their listings independently, improving scalability and business potential.

Security can be strengthened by implementing features such as **Two-Factor Authentication (2FA)**, advanced encryption techniques, and secure API handling.

The system can include **advanced analytics and reporting tools**, such as interactive dashboards and graphical reports, to analyze booking trends, user behavior, and revenue insights.

Integration of **multiple payment gateways**, including international options, digital wallets, UPI, and net banking, can improve flexibility and convenience for users.

Adding **multilingual support** will make the system accessible to users from different regions and language backgrounds.

The implementation of **voice search and AI-based assistants** can enhance usability and provide a more interactive experience.

The system can also include **real-time location tracking** using map services to improve navigation and property discovery.

CONCLUSION

The **StayFinder – Smart Hostel & Room Finder System** successfully addresses the challenges associated with traditional methods of finding rental accommodations. It provides a **centralized, efficient, and user-friendly platform** that simplifies the process of searching, booking, and managing rental properties.

The system integrates modern technologies such as the **MERN stack** along with **Artificial Intelligence**, enabling advanced features like AI-based rent prediction, real-time communication, secure payment processing, and automated rental agreement generation.

It enhances transparency by helping users identify fair pricing and reduces manual effort through automation. The system also improves communication between tenants and property owners through real-time interaction. Additionally, the inclusion of a roommate matching feature provides an innovative solution for shared accommodation.

The project was developed during an internship at **Sumago Infotech Pvt. Ltd.**, which provided valuable industry exposure and practical experience in full-stack development and system design.

Although the system has certain limitations, it provides a strong foundation for future enhancements such as mobile application development, advanced analytics, and improved AI capabilities.

In conclusion, the StayFinder system is a **scalable, reliable, and intelligent solution** that meets modern user requirements and contributes to the digital transformation of the rental accommodation process.

BIBLIOGRAPHY

1. W3Schools – <https://www.w3schools.com>
2. GeeksforGeeks – <https://www.geeksforgeeks.org>
3. MDN Web Docs – <https://developer.mozilla.org>
4. Stack Overflow – <https://stackoverflow.com>
5. React.js Documentation – <https://react.dev>
6. Node.js Documentation – <https://nodejs.org>
7. Scikit-learn Documentation – <https://scikit-learn.org>
8. FastAPI Documentation – <https://fastapi.tiangolo.com>
9. PDFKit Documentation – <https://pdfkit.org>